

P3070R1

Formatting enums

Published Proposal, 2024-10-13

Author:

[Victor Zverovich](#)

Audience:

LEWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Table of Contents

1 Introduction

2 Changes since R0

3 SG16 Poll

4 Motivation and Scope

5 Proposed Change

6 Impact on the Standard

7 Implementation

Appendix A: Benchmark

References

Informative References

"It is a mistake to think you can solve any major problems just with potatoes." — Douglas Adams

§ 1. Introduction

`std::format`, as introduced in C++20, has significantly improved string formatting in C++.

However, custom formatting for enumeration types currently requires creating somewhat verbose formatter specializations. This proposal aims to introduce a more intuitive and simpler method to define custom formats for enums using `format_as`. When formatting enums as integers it is also more efficient than a formatter specialization.

§ 2. Changes since R0

- Added the SG16 poll results.

§ 3. SG16 Poll

Poll 3: Forward P3070R0 to LEWG.

No objection to unanimous consent.

§ 4. Motivation and Scope

Enums are fundamental in C++ for representing sets of named constants. There is often a need to convert these enums to string representations, especially for logging, debugging, or interfacing with users. The current methods for customizing enum formatting in `std::format` are not as user-friendly or integrated as they could be.

With the introduction of a `format_as` extension for enums, we aim to:

- Simplify the process of defining custom formatting representations for enums.
- Improve enum formatting efficiency.
- Ensure compatibility with existing code and minimal disruption.

Consider the following example:

```
namespace kevin_namespacy {  
enum class film {  
    house_of_cards, american_beauty, se7en = 7  
};  
}
```

```
};  
}
```

If we want to format this enum as an underlying type with `std::format` we have two options. The first option is defining a formatter specialization:

```
template <>  
struct std::formatter<kevin_namespacy::film> : formatter<int> {  
    auto format(kevin_namespacy::film f, format_context& ctx) {  
        return formatter<int>::format(std::to_underlying(f), ctx);  
    }  
};
```

The drawback of this option is that even with forwarding to another formatter there is a fair amount of boilerplate and it cannot be done in the same namespace.

The second option is converting the enum to the underlying type:

```
film f = kevin_namespacy::se7en;  
auto s = std::format("{} ", std::to_underlying(f));
```

The drawback of the second option is that the conversion has to be done at every call site.

§ 5. Proposed Change

We propose adding a `format_as` extension point to `std::format`. `format_as` is a function discovered by the argument-dependent lookup (ADL) that takes an enum to be formatted as the argument and converts it to an object of another formattable type, normally an integer or a string.

This significantly improves user experience by eliminating almost all boilerplate:

Before:

```
namespace kevin_namespacy {  
    enum class film {...};  
}  
template <>  
struct std::formatter<kevin_namespacy::film> : formatter<int> {  
    auto format(kevin_namespacy::film f, format_context& ctx) const {
```

```

    return formatter<int>::format(std::to_underlying(f), ctx);
}
};

```

After:

```

namespace kevin_namespacy {
enum class film {...};
auto format_as(film f) { return std::to_underlying(f); }
}

```

The semantics of `format_as` is the same as the corresponding "forwarding" `formatter` specialization.

`format_as` can be used to format enums as strings as well:

```

enum class color {red, green, blue};

auto format_as(color c) -> std::string_view {
    switch (c) {
        case color::red:    return "red";
        case color::green:  return "green";
        case color::blue:   return "blue";
    }
}

auto s = std::format("{} ", color::red); // s == "red "

```

Apart from usability improvement, if the target type is one of the built-in types directly supported by `std::format`, formatting can be implemented more efficiently. Instead of going through the general-purpose `formatter` API the enum can be converted directly to the built-in type at the call site. And conversion from an enum to its underlying type is effectively a noop so there is no effect on the binary size.

The difference can be seen on the following benchmark results for an enum similar to `std::byte`:

Benchmark	Time	CPU	Iterations
-----------	------	-----	------------

BM_Formatter	17.7 ns	17.7 ns	38037070
BM_FormatAs	8.90 ns	8.88 ns	79036210

Considering that `format_as` has almost 2x better performance, this paper also proposes making `std::byte` formattable using the new facility.

§ 6. Impact on the Standard

This proposal is an additive change to the existing `<format>` standard library component and does not necessitate alterations to current language features or core library interfaces. It is a backward-compatible enhancement that addresses a common use case in `std::format`.

§ 7. Implementation

The proposed extension API has been implemented in the open-source `{fmt}` library ([\[FMT\]](#)) and as of December 2023 has been shipping for two major versions. It has been recently extended to all user-defined types and not just enums but this is not proposed in the current paper since usage experience is still limited.

§ Appendix A: Benchmark

This appendix gives the source code of the benchmark used for comparing performance of `format_as` with a `formatter` specialization.

```
#include <benchmark/benchmark.h>
#include <fmt/core.h>

enum class byte_for_formatter : unsigned char {};

template <>
struct fmt::formatter<byte_for_formatter> : fmt::formatter<unsigned char> {
    auto format(byte_for_formatter b, fmt::format_context& ctx) {
        return fmt::formatter<unsigned char>::format(
            static_cast<unsigned char>(b), ctx);
    }
};
```

```

enum class byte_for_format_as : unsigned char {};

auto format_as(byte_for_format_as b) { return static_cast<unsigned char>(b); }

static void BM_Formatter(benchmark::State& state) {
    auto b = byte_for_formatter();
    for (auto _ : state) {
        std::string formatted = fmt::format("{} ", b);
        benchmark::DoNotOptimize(formatted);
    }
}
BENCHMARK(BM_Formatter);

static void BM_FormatAs(benchmark::State& state) {
    auto b = byte_for_format_as();
    for (auto _ : state) {
        std::string formatted = fmt::format("{} ", b);
        benchmark::DoNotOptimize(formatted);
    }
}
BENCHMARK(BM_FormatAs);

BENCHMARK_MAIN();

```

§ References

§ Informative References

[FMT]

Victor Zverovich; et al. *The {fmt} library*. URL: <https://github.com/fmtlib/fmt>